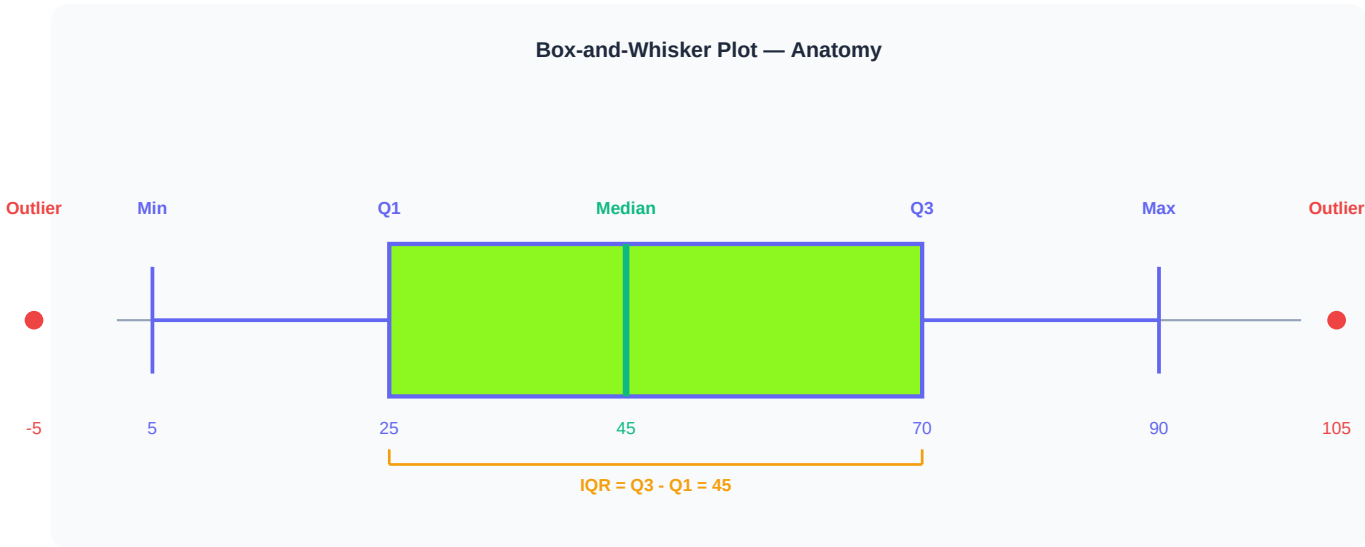


COMPLETE MASTER GUIDE

# Interquartile Range Analysis & Box-Whisker Diagrams in SQL

Outlier Detection · Fence Calculations · Window Functions  
Percentile Patterns · Multi-group Analysis · Edge Cases & Best Practices



|                      |                                                           |
|----------------------|-----------------------------------------------------------|
| Skill Level          | Intermediate → Advanced                                   |
| SQL Dialects Covered | PostgreSQL · MySQL 8+ · SQL Server · BigQuery · Snowflake |
| Topics               | 14 Deep-Dive Sections · 40+ Annotated Queries             |

# Table of Contents

|    |                                                   |    |
|----|---------------------------------------------------|----|
| 1. | Foundations — What is IQR and Why it Matters      | 3  |
| 2. | Anatomy of a Box-and-Whisker Plot                 | 4  |
| 3. | Calculating IQR in SQL — Core Patterns            | 5  |
| 4. | Identifying Outliers with Fence Method            | 7  |
| 5. | Excluding Outliers — Filtered Analysis Patterns   | 9  |
| 6. | Window Functions for IQR Analysis                 | 11 |
| 7. | Multi-Group / Partitioned IQR Analysis            | 12 |
| 8. | Percentile Techniques Across SQL Dialects         | 14 |
| 9. | Whisker Diagram Data Preparation in SQL           | 15 |
| 1  |                                                   |    |
| 0. | Handling Skewed Distributions & Edge Cases        | 17 |
| 1  |                                                   |    |
| 1. | Advanced Patterns — Modified Z-Score & Robust IQR | 18 |
| 1  |                                                   |    |
| 2. | Performance Optimization                          | 19 |
| 1  |                                                   |    |
| 3. | Common Challenges & Troubleshooting               | 20 |
| 1  |                                                   |    |
| 4. | Quick-Reference Cheat Sheet                       | 22 |

# 1 Foundations — What is IQR and Why it Matters

The **Interquartile Range (IQR)** is the distance between the 25th percentile (Q1) and the 75th percentile (Q3) of a dataset. It captures the middle 50% of your data and is the backbone of the box-and-whisker (box plot) visualization.

| Formula                             | Description                                                  |
|-------------------------------------|--------------------------------------------------------------|
| $IQR = Q3 - Q1$                     | Spread of the central half of data                           |
| Lower Fence = $Q1 - 1.5 \times IQR$ | Minimum threshold — values below are outliers                |
| Upper Fence = $Q3 + 1.5 \times IQR$ | Maximum threshold — values above are outliers                |
| Mild Outlier                        | Beyond $1.5 \times IQR$ but within $3 \times IQR$ from Q1/Q3 |
| Extreme Outlier                     | Beyond $3 \times IQR$ from Q1 or Q3 (Tukey outer fences)     |

## Why use IQR instead of Mean ± StdDev?

| Property                    | Mean ± Std Dev                       | IQR Method                       |
|-----------------------------|--------------------------------------|----------------------------------|
| Robustness to outliers      | ■ Sensitive                          | ■ Resistant                      |
| Assumes normal distribution | Yes                                  | No                               |
| Works on skewed data        | ■ Poor                               | ■ Good                           |
| Interpretability            | Moderate                             | High                             |
| Outlier definition          | Arbitrary ( $2\sigma$ or $3\sigma$ ) | Data-driven ( $1.5 \times IQR$ ) |

■ **TIP:** IQR is a non-parametric measure — it makes NO assumption about the underlying distribution. This makes it the preferred choice for real-world business data (sales, latencies, salaries).

## 2 Anatomy of a Box-and-Whisker Plot

Every component of a box plot maps to a precise statistical concept. Understanding each element is critical before writing SQL queries.

| Component            | Statistical Meaning       | SQL Equivalent                                                  |
|----------------------|---------------------------|-----------------------------------------------------------------|
| Left whisker end     | Minimum non-outlier value | <code>MIN(val) WHERE val &gt;= lower_fence</code>               |
| Box left edge (Q1)   | 25th percentile           | <code>PERCENTILE_CONT(0.25)</code>                              |
| Center line (Median) | 50th percentile           | <code>PERCENTILE_CONT(0.50)</code>                              |
| Box right edge (Q3)  | 75th percentile           | <code>PERCENTILE_CONT(0.75)</code>                              |
| Right whisker end    | Maximum non-outlier value | <code>MAX(val) WHERE val &lt;= upper_fence</code>               |
| Dots beyond whiskers | Outliers                  | <code>WHERE val &lt; lower_fence OR val &gt; upper_fence</code> |
| Box width (IQR)      | Middle 50% spread         | <code>Q3 - Q1</code>                                            |
| Whisker length       | Data range excl. outliers | <code>fence_max - fence_min</code>                              |

### Key Insight: What the Box Tells You

- **Box position (left or right of center)** → Distribution skewness
- **Long whiskers relative to box** → High variability in tails
- **Many outlier dots** → Heavy-tailed distribution
- **Short box, long whiskers** → Concentrated center, spread extremes
- **Median near Q1** → Right-skewed (positive skew)
- **Median near Q3** → Left-skewed (negative skew)

## 3 Calculating IQR in SQL — Core Patterns

### Pattern 1: PostgreSQL / Redshift / BigQuery (PERCENTILE\_CONT)

```
-- Pattern 1: Standard ANSI SQL (PostgreSQL, BigQuery, Redshift, Snowflake)

SELECT
  PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1,
  PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY salary) AS median,
  PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3,
  PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
  - PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS iqr
FROM employees;
```

### Pattern 2: MySQL 8+ (NTILE or Subquery Approach)

```
-- MySQL 8+: Use window functions with ROW_NUMBER to find quartiles

WITH ranked AS (
  SELECT salary,
    ROW_NUMBER() OVER (ORDER BY salary) AS rn,
    COUNT(*) OVER () AS total_rows
  FROM employees
),
quartiles AS (
  SELECT
    AVG(CASE WHEN rn IN (FLOOR(total_rows*0.25), CEIL(total_rows*0.25))
      THEN salary END) AS q1,
    AVG(CASE WHEN rn IN (FLOOR(total_rows*0.75), CEIL(total_rows*0.75))
      THEN salary END) AS q3
  FROM ranked
)
SELECT q1, q3, (q3 - q1) AS iqr FROM quartiles;
```

### Pattern 3: SQL Server (PERCENTILE\_CONT as Window Function)

```
-- SQL Server 2012+: PERCENTILE_CONT is a window function (not aggregate)

WITH stats AS (
  SELECT salary,
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)
    OVER () AS q1,
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
```

```
OVER () AS q3
FROM employees
)
SELECT DISTINCT q1, q3, (q3 - q1) AS iqr,
q1 - 1.5*(q3-q1) AS lower_fence,
q3 + 1.5*(q3-q1) AS upper_fence
FROM stats;
```

■ **WARNING:** SQL Server's PERCENTILE\_CONT requires an OVER() clause — it is a window function, NOT an aggregate. Always use SELECT DISTINCT or a CTE to avoid row duplication.

## 4 Identifying Outliers with the Fence Method

Tukey's fence method defines outlier boundaries mathematically. There are two fence levels — **inner fences** ( $1.5 \times \text{IQR}$ ) for mild outliers and **outer fences** ( $3 \times \text{IQR}$ ) for extreme outliers.

### Full Outlier Classification Query

```
-- Complete outlier tagging using fence method (PostgreSQL)

WITH quartiles AS (
  SELECT
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1,
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3
  FROM employees
),
fences AS (
  SELECT
    q1, q3, (q3 - q1) AS iqr,
    q1 - 1.5*(q3 - q1) AS inner_lower,
    q3 + 1.5*(q3 - q1) AS inner_upper,
    q1 - 3.0*(q3 - q1) AS outer_lower,
    q3 + 3.0*(q3 - q1) AS outer_upper
  FROM quartiles
)
SELECT
  e.employee_id, e.name, e.salary,
  CASE
    WHEN e.salary < f.outer_lower OR e.salary > f.outer_upper
    THEN 'EXTREME_OUTLIER'
    WHEN e.salary < f.inner_lower OR e.salary > f.inner_upper
    THEN 'MILD_OUTLIER'
    ELSE 'NORMAL'
  END AS outlier_status
FROM employees e, fences f
ORDER BY e.salary;
```

### Fence Summary Table

| Fence Type    | Lower Bound                  | Upper Bound                  | Classification  |
|---------------|------------------------------|------------------------------|-----------------|
| Inner (Tukey) | $Q1 - 1.5 \times \text{IQR}$ | $Q3 + 1.5 \times \text{IQR}$ | Mild Outlier    |
| Outer (Tukey) | $Q1 - 3.0 \times \text{IQR}$ | $Q3 + 3.0 \times \text{IQR}$ | Extreme Outlier |

|                        |                       |                       |                       |
|------------------------|-----------------------|-----------------------|-----------------------|
| Custom (Conservative)  | $Q1 - 2.0 \times IQR$ | $Q3 + 2.0 \times IQR$ | Domain-specific       |
| Symmetric (Percentile) | P5 (5th pct)          | P95 (95th pct)        | Percentile-based trim |

■ **TIP:** The 1.5×IQR rule flags ~0.7% of data as outliers in a normal distribution. For business data, consider adjusting to 2.0×IQR to be less aggressive.

## 5 Excluding Outliers — Filtered Analysis Patterns

### Pattern A: Simple WHERE Filter (Inline Subquery)

```
-- Compute clean metrics after excluding outliers

SELECT
COUNT(*) AS clean_count,
AVG(salary) AS clean_avg,
STDDEV(salary) AS clean_stddev,
MIN(salary) AS clean_min,
MAX(salary) AS clean_max
FROM employees
WHERE salary BETWEEN (
(SELECT PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)
FROM employees) - 1.5 * (
(SELECT PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
FROM employees)
- (SELECT PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)
FROM employees))
) AND (
(SELECT PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
FROM employees) + 1.5 * (
(SELECT PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
FROM employees)
- (SELECT PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)
FROM employees)))
);
```

### Pattern B: Reusable CTE Approach (Recommended)

```
-- BEST PRACTICE: Calculate fences once, reuse everywhere

WITH
bounds AS (
SELECT
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1,
PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY salary) AS median,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
- PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS iqr
```

```
FROM employees
),
fenced AS (
SELECT
q1, q3, iqr, median,
q1 - 1.5 * iqr AS lower_fence,
q3 + 1.5 * iqr AS upper_fence
FROM bounds
),
clean_data AS (
SELECT e.*
FROM employees e
CROSS JOIN fenced f
WHERE e.salary BETWEEN f.lower_fence AND f.upper_fence
)
SELECT
COUNT(*) AS n,
ROUND(AVG(salary), 2) AS trimmed_mean,
ROUND(STDDEV(salary), 2) AS trimmed_stddev
FROM clean_data;
```

## 6 Window Functions for IQR Analysis

Window functions allow you to compute IQR **per row**, enabling row-level outlier flags without GROUP BY aggregation losing your detail records.

```
-- Row-level outlier flag using window functions (PostgreSQL/Snowflake)
SELECT
employee_id, name, department, salary,
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)
OVER () AS q1,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
OVER () AS q3,
CASE
WHEN salary < PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) OVER()
- 1.5*(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) OVER())
-PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) OVER())
THEN 'LOW_OUTLIER'
WHEN salary > PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) OVER()
+ 1.5*(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) OVER())
-PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) OVER())
THEN 'HIGH_OUTLIER'
ELSE 'NORMAL'
END AS outlier_flag
FROM employees;
```

■ **TIP:** Use a CTE to compute Q1, Q3 once, then JOIN back — it avoids repeating the PERCENTILE\_CONT expression 6+ times and makes queries readable and maintainable.

## 7 Multi-Group / Partitioned IQR Analysis

Real-world analysis requires IQR computed **per category** (department, region, product). Use PARTITION BY to isolate each group's distribution independently.

### Pattern: Per-Department Outlier Detection

```
-- Each department gets its own IQR -- prevents cross-group contamination
WITH dept_stats AS (
  SELECT
    employee_id, name, department, salary,
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)
    OVER (PARTITION BY department) AS dept_q1,
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
    OVER (PARTITION BY department) AS dept_q3
  FROM employees
),
flagged AS (
  SELECT *,
    dept_q3 - dept_q1 AS dept_iqr,
    dept_q1 - 1.5*(dept_q3 - dept_q1) AS dept_lower,
    dept_q3 + 1.5*(dept_q3 - dept_q1) AS dept_upper
  FROM dept_stats
)
SELECT
  department, name, salary,
  ROUND(dept_q1, 2) AS q1,
  ROUND(dept_q3, 2) AS q3,
  ROUND(dept_iqr, 2) AS iqr,
  ROUND(dept_lower, 2) AS lower_fence,
  ROUND(dept_upper, 2) AS upper_fence,
  CASE WHEN salary < dept_lower OR salary > dept_upper
  THEN 'OUTLIER' ELSE 'NORMAL' END AS status
FROM flagged
ORDER BY department, salary;
```

■ **WARNING:** CRITICAL: Never compute a single global IQR when groups have different distributions (e.g., Engineering vs. Support salaries). Always partition per business group to avoid false outlier classification.

### Aggregated Box Plot Summary Per Group

```

-- Summary table for rendering multiple box plots (one per department)
SELECT
department,
COUNT(*) AS n,
MIN(salary) AS raw_min,
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1,
PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY salary) AS median,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3,
MAX(salary) AS raw_max,
-- Whisker endpoints (non-outlier min/max)
MIN(CASE WHEN salary >= (PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)
- 1.5*(PERCENTILE_CONT(0.75) WITHIN GROUP
(ORDER BY salary)
- PERCENTILE_CONT(0.25) WITHIN GROUP
(ORDER BY salary)))
THEN salary END) AS whisker_low,
MAX(CASE WHEN salary <= (PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
+ 1.5*(PERCENTILE_CONT(0.75) WITHIN GROUP
(ORDER BY salary)
- PERCENTILE_CONT(0.25) WITHIN GROUP
(ORDER BY salary)))
THEN salary END) AS whisker_high
FROM employees
GROUP BY department
ORDER BY department;

```

## 8 Percentile Techniques Across SQL Dialects

| Dialect    | Function                                               | Type        | Notes                       |
|------------|--------------------------------------------------------|-------------|-----------------------------|
| PostgreSQL | PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY col)         | Aggregate   | Interpolates between values |
| PostgreSQL | PERCENTILE_DISC(p) WITHIN GROUP (ORDER BY col)         | Aggregate   | Returns actual data value   |
| BigQuery   | PERCENTILE_CONT(col, p) OVER ()                        | Window      | Must use OVER clause        |
| Snowflake  | PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY col)         | Aggregate   | Identical to PostgreSQL     |
| SQL Server | PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY col) OVER () | Window only | Requires OVER()             |
| MySQL 8+   | No native — use NTILE or manual ROW_NUMBER             | N/A         | See Pattern 2               |
| SQLite     | No native — use subquery approach                      | N/A         | Use CASE + ROW_NUMBER       |
| Hive       | PERCENTILE(col, p) or PERCENTILE_APPROX                | Aggregate   | APPROX for large data       |

### CONT vs DISC — When to Use Which

```
-- PERCENTILE_CONT: interpolates (good for continuous data like salary, weight)
-- Returns 62.5 for dataset [50, 60, 70, 80] at 0.25
SELECT PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1_cont
FROM employees;

-- PERCENTILE_DISC: picks actual value (good for discrete data: scores, counts)
-- Returns 60 for dataset [50, 60, 70, 80] at 0.25
SELECT PERCENTILE_DISC(0.25) WITHIN GROUP (ORDER BY score) AS q1_disc
FROM test_scores;
```

## 9 Whisker Diagram Data Preparation in SQL

To render a box plot in any BI tool (Tableau, Power BI, Grafana, custom charts), you need to output exactly these columns from SQL:

| Column        | Meaning                     | Formula                           |
|---------------|-----------------------------|-----------------------------------|
| group_label   | Category label              | department / product / region     |
| whisker_low   | Left whisker tip            | MIN(val) WHERE val >= lower_fence |
| q1            | Box left edge               | PERCENTILE_CONT(0.25)             |
| median        | Box center line             | PERCENTILE_CONT(0.50)             |
| q3            | Box right edge              | PERCENTILE_CONT(0.75)             |
| whisker_high  | Right whisker tip           | MAX(val) WHERE val <= upper_fence |
| outliers      | JSON array or separate rows | WHERE val < lower OR val > upper  |
| n             | Sample size                 | COUNT(*)                          |
| outlier_count | Number of outliers          | COUNT(*) WHERE flagged            |

### Complete Box Plot Export Query

```
-- Full box plot data export with outlier list (PostgreSQL)
WITH stats AS (
SELECT
department AS group_label,
COUNT(*) AS n,
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1,
PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY salary) AS median,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
- PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS iqr
FROM employees
GROUP BY department
),
fenced AS (
SELECT *,
q1 - 1.5*iqr AS lower_fence,
q3 + 1.5*iqr AS upper_fence
FROM stats
),
whiskers AS (
SELECT
```

```

f.group_label, f.n, f.q1, f.median, f.q3, f.iqr,
f.lower_fence, f.upper_fence,
MIN(CASE WHEN e.salary >= f.lower_fence THEN e.salary END) AS whisker_low,
MAX(CASE WHEN e.salary <= f.upper_fence THEN e.salary END) AS whisker_high,
COUNT(CASE WHEN e.salary < f.lower_fence OR e.salary > f.upper_fence
THEN 1 END) AS outlier_count
FROM fenced f
JOIN employees e ON e.department = f.group_label
GROUP BY f.group_label, f.n, f.q1, f.median, f.q3,
f.iqr, f.lower_fence, f.upper_fence
)
SELECT
group_label, n, outlier_count,
ROUND(whisker_low, 2) AS whisker_low,
ROUND(q1, 2) AS q1,
ROUND(median, 2) AS median,
ROUND(q3, 2) AS q3,
ROUND(whisker_high, 2) AS whisker_high,
ROUND(iqr, 2) AS iqr,
ROUND(lower_fence, 2) AS lower_fence,
ROUND(upper_fence, 2) AS upper_fence
FROM whiskers
ORDER BY group_label;

```

## 10 Handling Skewed Distributions & Edge Cases

| Challenge                | Symptom                              | SQL Solution                                    |
|--------------------------|--------------------------------------|-------------------------------------------------|
| IQR = 0 (all same value) | Fence = Q1 = Q3, all data = outliers | Add NULLIF(iqr,0) guard; use fallback range     |
| Tiny groups (n < 4)      | Percentile = raw value, unstable     | Filter: HAVING COUNT(*) >= 10 before IQR        |
| Highly skewed right      | Upper fence too tight, over-flags    | Use log(salary) for IQR, back-transform         |
| Negative values          | Lower fence may be absurd negative   | Cap at domain minimum: GREATEST(lower_fence, 0) |
| NULLs in data            | Silently ignored by PERCENTILE_CONT  | Always check: WHERE col IS NOT NULL             |
| Tied values at Q1/Q3     | CONT vs DISC give different fences   | Document which you use; be consistent           |
| Temporal drift           | IQR shifts over time                 | Compute IQR per time window (monthly)           |

### IQR = 0 Guard Pattern

```
-- Safe fence calculation when IQR could be zero

WITH q AS (
  SELECT
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY val) AS q1,
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY val) AS q3
  FROM my_table
)
SELECT
  q1, q3,
  CASE
    WHEN q3 - q1 = 0 THEN 'IQR is zero - all values identical'
    ELSE CAST(q1 - 1.5*(q3-q1) AS TEXT)
  END AS lower_fence_safe
FROM q;
```

## 11 Advanced Patterns — Modified Z-Score & Robust IQR

### Modified Z-Score (MAD Method)

The **Median Absolute Deviation (MAD)** is even more robust than IQR for datasets with many outliers, as it uses the median of deviations rather than percentile differences.

```
-- Modified Z-Score using Median Absolute Deviation (PostgreSQL)
WITH med AS (
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median
FROM employees
),
deviations AS (
SELECT salary, ABS(salary - m.median) AS abs_dev
FROM employees, med m
),
mad_val AS (
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY abs_dev) AS mad
FROM deviations
)
-- Modified Z = 0.6745 * (x - median) / MAD |z| > 3.5 = outlier
SELECT
e.employee_id, e.salary,
ROUND(0.6745 * ABS(e.salary - m.median) / NULLIF(mv.mad, 0), 3)
AS modified_z_score,
CASE WHEN 0.6745 * ABS(e.salary - m.median) / NULLIF(mv.mad, 0) > 3.5
THEN 'OUTLIER' ELSE 'NORMAL' END AS status
FROM employees e, med m, mad_val mv;
```

### Winsorization (Capping Instead of Removing)

```
-- Winsorize: cap outliers at fence values instead of dropping them
WITH fences AS (
SELECT
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) - 1.5*(
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
- PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)) AS lo,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) + 1.5*(
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
- PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)) AS hi
```

```
FROM employees
)
SELECT
employee_id, salary AS original_salary,
GREATEST(f.lo, LEAST(f.hi, salary)) AS winsorized_salary
FROM employees, fences f;
```

## 12 Performance Optimization

| Technique                             | When to Use                     | Impact                              |
|---------------------------------------|---------------------------------|-------------------------------------|
| Compute IQR once in CTE, CROSS JOIN   | Always — default pattern        | Avoids N repeated percentile passes |
| PERCENTILE_APPROX (Hive/BigQuery)     | Tables > 100M rows              | 10-100x faster, ~1% error           |
| Pre-aggregate to daily/weekly buckets | Time-series with high frequency | Reduces cardinality before IQR      |
| Materialized views for IQR bounds     | Dashboard queries run often     | Compute once, serve many            |
| Index on the value column             | Tables < 50M rows, OLTP         | Speeds sort-dependent percentile    |
| TABLESAMPLE for exploration           | Quick EDA on massive tables     | Sample 1-10%, fast approximate IQR  |

### Approximate IQR for Large Data (BigQuery)

```
-- BigQuery: APPROX_QUANTILES for fast approximate percentiles
WITH approx AS (
  SELECT APPROX_QUANTILES(salary, 100) AS pcts
  FROM `project.dataset.employees`
)
SELECT
  pcts[OFFSET(25)] AS q1,
  pcts[OFFSET(50)] AS median,
  pcts[OFFSET(75)] AS q3,
  pcts[OFFSET(75)] - pcts[OFFSET(25)] AS approx_iqr
FROM approx;
```

## 13 Common Challenges & Troubleshooting

| ■ Problem                                                | ■ Fix                                                                                                                                |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| "All rows flagged as outliers"                           | IQR = 0. Use <code>NULLIF(iqr, 0)</code> . Check if data has very low variance.                                                      |
| "Fence produces negative value for positive metric"      | Clamp: <code>GREATEST(lower_fence, 0)</code> for money/counts.                                                                       |
| "Different tools give different Q1/Q3"                   | Interpolation method differs (R has 9 types). <code>PERCENTILE_CONT</code> = Type 7.                                                 |
| "Outlier count jumps month-over-month"                   | Using global IQR. Switch to rolling 30-day IQR window per group.                                                                     |
| "Query too slow on 500M rows"                            | Use <code>APPROX_QUANTILES</code> or <code>NTILE(4)</code> . Pre-aggregate.                                                          |
| "Whisker shorter than expected"                          | Check <code>BETWEEN</code> is inclusive. Use <code>&gt;=</code> and <code>&lt;=</code> not <code>&gt;</code> and <code>&lt;</code> . |
| "NULL rows removed from clean set incorrectly"           | <code>salary BETWEEN NULL AND x</code> evaluates to <code>NULL</code> = excluded. Add <code>IS NOT NULL</code> .                     |
| "SQL Server: no aggregate <code>PERCENTILE_CONT</code> " | Add <code>OVER()</code> clause. SQL Server requires window function context.                                                         |

## 14 Quick-Reference Cheat Sheet

### The 5 Must-Know IQR Formulas

| # | Name              | Formula               |
|---|-------------------|-----------------------|
| 1 | IQR               | $Q3 - Q1$             |
| 2 | Lower Inner Fence | $Q1 - 1.5 \times IQR$ |
| 3 | Upper Inner Fence | $Q3 + 1.5 \times IQR$ |
| 4 | Lower Outer Fence | $Q1 - 3.0 \times IQR$ |
| 5 | Upper Outer Fence | $Q3 + 3.0 \times IQR$ |

### Essential Query Skeleton (PostgreSQL)

```
-- ★ MASTER TEMPLATE – copy and adapt for any IQR task ★  
  
WITH  
  
-- Step 1: Compute quartiles  
quartiles AS (  
SELECT  
[group_col],  
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY [value_col]) AS q1,  
PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY [value_col]) AS median,  
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY [value_col]) AS q3  
FROM [your_table]  
[WHERE [value_col] IS NOT NULL]  
[GROUP BY [group_col]]  
) ,  
  
-- Step 2: Derive IQR and fences  
fences AS (  
SELECT *,  
q3 - q1 AS iqr,  
q1 - 1.5 * (q3 - q1) AS lower_fence,  
q3 + 1.5 * (q3 - q1) AS upper_fence  
FROM quartiles  
) ,  
  
-- Step 3: Join back and classify rows  
classified AS (  
SELECT  
t.*,  
f.q1, f.median, f.q3, f.iqr, f.lower_fence, f.upper_fence,
```

```
CASE
WHEN t.[value_col] < f.lower_fence THEN 'LOW_OUTLIER'
WHEN t.[value_col] > f.upper_fence THEN 'HIGH_OUTLIER'
ELSE 'NORMAL'
END AS outlier_status
FROM [your_table] t
JOIN fences f ON t.[group_col] = f.[group_col]
)

-- Step 4: Use as needed
SELECT * FROM classified WHERE outlier_status = 'NORMAL'; -- clean data

-- OR
SELECT * FROM classified WHERE outlier_status != 'NORMAL'; -- outliers only
```

### Decision Guide: Which Pattern to Use

| Situation                        | Pattern                                      |
|----------------------------------|----------------------------------------------|
| Simple global IQR, one metric    | Section 3, Pattern 1 (CTE + PERCENTILE_CONT) |
| Flag rows without aggregating    | Section 6 (Window Function)                  |
| Different groups, separate IQRs  | Section 7 (PARTITION BY department)          |
| Export data for box plot chart   | Section 9 (Full Box Plot Export Query)       |
| Data has many duplicates / IQR=0 | Section 10 (IQR=0 Guard Pattern)             |
| Extremely skewed or heavy-tailed | Section 11 (MAD / Modified Z-Score)          |
| Cap instead of remove outliers   | Section 11 (Winsorization)                   |
| Table > 100M rows, need speed    | Section 12 (APPROX_QUANTILES)                |
| MySQL without PERCENTILE_CONT    | Section 3, Pattern 2 (ROW_NUMBER)            |
| SQL Server dialect               | Section 3, Pattern 3 (OVER() required)       |

■ **TIP:** Master the CTE pattern in Section 5 Pattern B first — it is the foundation for 90% of real-world IQR work. Everything else is a variation of that skeleton.